

SOC Platform Technical Report

Phases 1, 2 and 3

Security Monitoring Foundation - Huawei Cloud Infrastructure - Backend Intelligence Layer

This report consolidates the technical architecture and logic of the SOC platform across three phases. It is designed for a PFE report, technical documentation, and presentation preparation.

Executive Summary

The project is built progressively. Phase 1 creates monitoring visibility with Wazuh, Suricata, Falco, Telegraf, Filebeat, Elasticsearch, and Kibana. Phase 2 provides Huawei Cloud infrastructure with ECS compute, private networking, security group controls, 100 GB storage, 128 GB RAM, 64 vCPU, and EIP 193.95.30.97. Phase 3 adds backend intelligence: alert processing, incident correlation, investigations, AI response, approval, Ansible execution, verification, archive, performance monitoring, IPS map, AI Assistant, and AI Operator.

Monitor -> Centralize -> Analyze -> Correlate -> Investigate -> Respond -> Verify -> Archive

Phase 1 - Security Monitoring Foundation

Objective

Phase 1 is the visibility foundation. It observes the infrastructure before backend intelligence and automation are added.

- Network visibility through Suricata.
- Host and endpoint visibility through Wazuh.
- Runtime and container visibility through Falco.
- Resource and infrastructure metrics through Telegraf.
- Log and alert forwarding through Filebeat and Falcossidekick.
- Central storage/search through Elasticsearch and visualization through Kibana.

Architecture and Data Flow

Source	Forwarding path	Destination
-----+-----+-----		
System logs	Filebeat	Elasticsearch
Authentication logs	Filebeat	Elasticsearch
Suricata events	eve.json -> Filebeat	Elasticsearch
Wazuh alerts	Filebeat	Elasticsearch
Falco runtime alerts	Falcossidekick	Elasticsearch
Telegraf metrics	Direct output	Elasticsearch
Elasticsearch data	Kibana	Dashboards / analysis

Deployment, Ports and Configurations

Service / Tool	Port / Configuration
-----+-----	
Elasticsearch	9200 HTTPS - /etc/elastic...
Kibana	5601 HTTPS - /etc/kibana/...
Wazuh Manager	1514 TCP events / 1515 TC...
Falcossidekick	2801 HTTP - docker-compos...
Filebeat	/etc/filebeat/filebeat.y...
Suricata	/etc/suricata/suricata.ya...
Telegraf	/etc/telegraf/telegraf.co...

Hardware Requirements

Minimum lab: 4 cores / 8 GB RAM / 50 GB disk
 Recommended: 8 cores / 16 GB RAM / 100 GB disk

Outcome

Phase 1 transforms isolated monitoring tools into a coherent visibility architecture. It creates the data foundation required for backend intelligence.

Phase 2 - Huawei Cloud Infrastructure

Objective

Phase 2 hosts the SOC stack on Huawei Cloud. It provides compute, storage, private networking, public access through an Elastic IP, and security group protection.

Cloud Infrastructure

Huawei Cloud Environment

- VPC / private network
- Private subnet
- Security group with controlled firewall rules
- Elastic IP: 193.95.30.97
- ECS virtual machine
 - Ubuntu Server
 - 64 vCPU
 - 128 GB RAM
 - 100 GB storage
- Hosted stack: Elasticsearch, Kibana, Wazuh, Suricata, Falco, Telegraf, Filebeat

Security Group Strategy

Service	Port	Recommendation
SSH	22	Restrict to admin IP
Kibana	5601	Restrict to analyst/VPN/a...
Elasticsearch	9200	Keep internal, not public
Wazuh events	1514	Allow only if external ag...
Wazuh register	1515	Allow only if external ag...
Falcosidekick	2801	Internal only

Outcome

Phase 2 provides the infrastructure backbone of the SOC platform and prepares a secure cloud base for the monitoring stack.

Phase 3 - Backend Intelligence Layer

Main Idea

Phase 3 converts raw Elasticsearch data into operational SOC intelligence. It normalizes alerts, correlates incidents, creates investigations, generates AI response plans, controls remediation execution, verifies results, archives evidence, and exposes data to the frontend.

Feature Map

Feature	Backend Entry	Frontend	Status
Performance Monitoring	api/routes/performance.py	/metrics	API Part B incomplete
IPS Map	api/routes/ips.py	/ips	Complete
AI Assistant	api/routes/assistant.py	/assistant	Frontend not verified
AI Operator	api/routes/operator.py	/operator	Frontend not verified
Alert Pipeline	pipeline/poller/main.py	/alerts	Complete
Incidents & Correlation	pipeline/datausage/incide...	/incidents	API truncated
Watcher & Investigations	response/watcher/main.py	/investigations	Complete
AI Response	response/ai_engine/main.py	/investigations	Complete

1. Alert Ingestion and Enrichment Pipeline

The alert pipeline polls Elasticsearch indices, normalizes source-specific documents, deduplicates, filters noise, enriches alerts, checks whitelist entries, tracks campaigns, persists locally, and can forward upstream.

```
ES raw documents -> poller -> mapper -> dedup -> filter -> enrichment -> campaign tracking ->
whitelist -> SQLite Alert
```

Main files:

- pipeline/poller/main.py: forwarder loop and cursor management
- pipeline/poller/alert_processor.py: process_single_alert pipeline
- pipeline/mappers/*.py: Wazuh / Suricata / Falco / Filebeat / generic mappers
- pipeline/enrichment/geoup.py: GeoIP enrichment
- pipeline/enrichment/mitrecraft.py: MITRE mapping
- pipeline/dedup.py: deduplication
- pipeline/noise_learner.py: auto-learned noise filtering

2. Local Storage and SOC Data Model

Table	Purpose
Alert	Clean normalized alerts
Incident	Grouped security cases
AlertIncidentLink	Links alerts to incidents...
Investigation	Investigation context and...
PlaybookApproval	Approval decisions
PlaybookRun	Execution results
FixVerification	Verification verdicts
Archive	Preserved resolved case h...

3. Incidents and Correlation

The incident module correlates related alerts into incidents using attack patterns, kill-chain progression, time-window clustering, shared observables, and MITRE overlap.

- Critical alerts create incidents.
- Known attack patterns create incidents.
- Kill chain with two or more MITRE phases creates incidents.
- Medium severity plus multiple recent alerts can create an incident.

4. Watcher and Investigations

The watcher looks for open incidents without investigations, creates Investigation rows, builds context, and triggers the AI engine.

- Linked alert timeline.
- IOCs: IPs, ports, services, domains, hashes, usernames, file paths.
- MITRE tactics and techniques.
- Behavioral indicators and authentication pattern analysis.
- Attack type and dynamic risk score.

5. AI Response Engine

The AI Response Engine generates investigation summaries, narratives, risk assessments, and Ansible playbooks. It uses circuit breaker protection, prompt building, LLM routing, structured parsing, storage, and auto-approval evaluation.

```
Investigation context -> circuit breaker -> prompt builder -> LLM -> parser -> storage ->
approval check
```

6. Approval, Execution, Verification and Archive

Generated remediation is controlled through approval gates, Ansible execution, fix verification, and archiving.

Verification outcome	Meaning
-----+-----	
likely_fixed	Remediation succeeded and...
not_fixed	Alerts or metrics continue
inconclusive	Evidence is unclear
playbook_failed_but_quiet	Playbook failed but sympt...

7. Performance Monitoring and Remediation

Performance monitoring polls Telegraf metrics from Elasticsearch, stores metrics/history/baselines in Redis, detects anomalies, determines root causes, creates performance investigations, generates dynamic playbooks, and verifies fixes.

- Metrics include CPU, memory, disk, network, load, procstat, netstat and disk_dir.
- Detection uses thresholds and statistical baselines.
- Cooldowns prevent flapping.
- Dynamic playbooks integrate with approval and Ansible execution.

8. IPS Attack Visualization

The IPS map converts alerts into geo-enriched attack events. It deduplicates, geolocates, categorizes attacks, derives lifecycle, and serves map data, live events, filters, and statistics.

- GeoIP includes country, city, coordinates, ASN and provider.
- Categories include brute force, web attack, reconnaissance, malware, DoS and C2.
- Lifecycle states include active, investigating, mitigated and blocked.

9. AI Assistant

The AI Assistant lets operators ask natural-language questions about SOC data. It extracts keywords, fetches related entities, ranks records, builds prompts, returns answers and sources, suggests actions, and stores conversations.

10. AI Operator

The AI Operator converts natural-language operational requests into controlled Ansible actions. It reasons about intent, matches templates, generates playbooks if needed, requires approval for mutating actions, executes Ansible, parses output, and returns structured analysis.

Example templates:

```
disk_usage, ram_usage, cpu_processes, open_ports, ssh_failures,
service_status, firewall_rules, docker_containers, file_read, package_check
```

Background Jobs and Reliability

Task	Role
-----+-----	
Alert Forwarder	Polls Elasticsearch and p...
Incident Correlation	Groups alerts into incide...
Incident Watcher	Creates investigations fr...
Performance Poller	Reads Telegraf metrics
Performance Monitoring	Detects anomalies and tri...
Retry Queue	Re-sends failed upstream ...
Watchdog	Logs memory and process h...
Backup	Copies DB, cursor state, ...

Known Gaps to Close

- api/routes/performance.py Part B is incomplete for disk-analysis, history and root-cause

endpoints.

- api/routes/incidents.py was truncated during delivery.
- Frontend pages for /metrics, /assistant and /operator need verification.
- api/routes/monitoring.py may need export if it exists.

Conclusion

Phase 1 creates visibility. Phase 2 provides the cloud infrastructure. Phase 3 transforms raw data into SOC intelligence and operations.

Monitor -> Centralize -> Analyze -> Correlate -> Investigate -> Respond -> Verify -> Archive

Diagram Appendix

All Mermaid diagrams are provided in the separate file SOC_Phase_Diagrams_Mermaid.md.

Source Documentation Used

- INDEX.md
- core_alert_pipeline.md
- core_incidents.md
- core_watcher.md
- core_ai_response.md
- performance_monitoring.md
- ips_map.md
- ai_assistant.md
- ai_operator.md